

Setting up Rumqttd: a Rust-based MQTT broker

I've been looking into building some WiFi-enabled embedded Rust hardware projects recently. When creating projects where the devices might need to communicate to each other, [MQTT](#) can be a really natural choice. MQTT is light-weight, and makes it easy to connect new devices without them needing to be directly aware of each other. Instead, your devices publish and subscribe to messages from an MQTT broker. While [there are a lot of brokers out there](#), I had a few requirements for the one I wanted to use for communicating with my embedded projects:

1. It needs to support self-hosting. My goal for most of my IoT projects is to keep them as isolated to my local network as possible. If I want anything I build to be accessible to me outside of my home network, I can use [Tailscale](#). I also want the services involved to be cheap/free if I can. Hardware is already an expensive hobby, so not paying for a bunch of services to improve it is ideal. Both of these factors mean that the many public or cloud-based MQTT brokers won't work for what I want.
2. I wanted it written in Rust (or at least a language I know well). I'm doing as much of the embedded development as I can in Rust. I wanted my MQTT broker to keep that pattern going. Keeping the languages limited to ones I have a good working knowledge of, I can make sure I feel comfortable debugging and contributing back to the project when things go wrong.

Luckily, I found [rumqttd](#) a Rust-based MQTT broker. It even has a corresponding MQTT client [rumqttdc](#) (though [rumqttdc](#) will not work for [no-std](#) Rust projects at this time, so it's not a great fit for most embedded use cases).

Setting up [rumqttd](#) using the container image

The way I've setup [rumqttd](#) locally is through the [Docker image](#) provided in the repository. The instructions in the repo suggest you can pull the image and tell it to use a default config file with a command like `docker run -p 1883:1883 -p 1884:1884 -it bytebeamio/rumqttd -c rumqttd.toml`. However, I've found

that this doesn't seem to work, causing `rumqttd` to panic. Instead, I recommend cloning and building the image locally.

1. Clone the `rumqtt` image from GitHub. I like to use the [GitHub CLI](#) where the command is `gh repo clone bytebeamio/rumqtt`, but you can use `git clone https://github.com/bytebeamio/rumqtt.git` to do the same thing.
2. Build the image locally (instructions here are for Docker, but any service that can build a Docker container should work). In the root of the `rumqtt` repo, run `docker build -t rumqttd .` (you can swap the `rumqttd` for whatever tag you want for the image).
3. When that's done, the command they suggest in the repo for use with the pre-built image should work (but with the image name swapped for what we called our image): `docker run -p 1883:1883 -p 1884:1884 -it rumqttd -c rumqttd.toml` (again swapping `rumqttd` for the tag you set in step 2). If everything works, ASCII art will spell out `RUMQTDD` in the terminal. If you connect an MQTT client to your local instance and subscribe/post through the server, your terminal will log info about each one. If you want to leave the container running in the background, you can use `Ctrl+p` then `Ctrl + q`. Otherwise you can stop the container with `Ctrl+c`.

If you don't want to use containers to run your server, I'd suggest following the instructions in the repo to get it up and running with either [cargo install](#) or [cargo run](#). I've not tested either of these methods, but am pretty confident that either option should work as long as you pass a config file.

Connecting clients

You'll note that in the Docker command above, 2 ports are exposed, 1883 and 1884. This is because the default config from the `rumqttd` repo sets up endpoints for 2 common MQTT versions: MQTT v4.1 on port 1883, and MQTT v5 on port 1884. So if you were connecting a client compatible with MQTT v4 from your computer running your broker, you'd point set the broker URL to `http://localhost:1883`. If you were doing the same but with an MQTT v5 client, you'd use `http://localhost:1884` for the broker.

Supporting both MQTT v4 & MQTT v5 allows your server to be compatible with most Rust-based MQTT clients. For instance, in `no-std` & `std` environments, [rust mqtt](#),

supports MQTT v5, but not v4. On the other hand, `rumqttd`, which only works in `std` environments, supports both MQTT v4 and MQTT v5, but the v4 implementation is more robust. Unfortunately, at the time of this writing, the MQTT client provided by `esp-rs/esp-idf-svc` is only compatible with MQTT v3. So if writing `std` applications for espressif chips, you should either look into a different MQTT client library, or find an MQTT broker that supports MQTT v3.

It's also worth calling out that the v4 & v5 brokers feature fully separate queues. So if you send a message to the v5 broker, clients listening to the v4 broker will not receive that message and vice-versa. You *might* be able to setup the bridge to relay messages across the versions, but I've not looked into it.

Updating the log level in a running broker

When writing custom clients and debugging connections, it can be useful to adjust the log level from the broker service is actively running. The default config provided in the repo provides an endpoint for this at port `:3030`. However, following the instructions from earlier, we're not yet exposing this endpoint outside of the container. To do that, we first need to add that port to our `docker run` command:

```
docker run -p 1883:1883 -p 1884:1884 -p 3030:3030 -it rumqttd -
```

After this, we can send a post request to `http://localhost:3030/logs` with a new log level at any time. e.g. if we wanted to get `trace` level logs, we could make request like this:

```
# sends a post request to localhost:3030 with a content-type of
curl -H "Content-Type: text/plain" -d "rumqttd=trace" http://lo
```

The docs suggest that you *can* send any valid `Tokio tracing-subscriber EnvFilter` value. However, for most of the debugging I need when testing MQTT clients, managing the `rumqttd` log levels is sufficient.

Since this is running only on my local network, I'm relatively comfortable leaving this config port constantly exposed while running my broker. If I were hosting my server on the public internet, I'd probably consider a different approach.

© Darrik Moberg — 2025

